

This technique works fine. However, coding it this way means that not only does every object instantiated from the `SpaceShip` class have to take the time to load the image, but each object takes up all the memory needed to represent a copy of the same image. Instead, we can have the class load the image once, and each `SpaceShip` object then uses the single image kept in the class, like this:

```
class SpaceShip():
    SPACE_SHIP_IMAGE = pygame.image.load('images/ship.png')
    def __init__(self, window, ...):
        self.image = pygwidgets.Image(window, (0, 0),
                                       SpaceShip.SPACE_SHIP_IMAGE)
```

An `Image` object (in `pygwidgets`, as used here) can use either a path to an image or an already loaded image. Allowing the class to load the image only *once* makes startup faster and results in lower memory usage.

Class Variables for Counting

A second way to use a class variable is to keep track of how many objects have been instantiated from a class. [Listing 11-3](#) shows an example.

File: ClassVariable.py

```
# Sample class

class Sample():
    ❶ nObjects = 0 # this is a class variable of the Sample class
    def __init__(self, name):
        self.name = name
        ❷ Sample.nObjects = Sample.nObjects + 1

    def howManyObjects(self):
        ❸ print('There are', Sample.nObjects, 'Sample objects')

    def __del__(self):
        ❹ Sample.nObjects = Sample.nObjects - 1

# Instantiate 4 objects
oSample1 = Sample('A')
oSample2 = Sample('B')
```